

# Data-Driven Optimization

과제 5 답안

## 과제 5 -1

수업시간에 진행한 실습 문제에 대하여 다음 질문에 답하여라.

1. 가변적 전기요금제도를 도입하지 않았을 경우 전기요금은 일률적으로 10 이라고 하자. 이때는 생산비용을 계산하여라.
2. 전기회사에서는 다음 3가지 요금제를 제안하였다. 이 회사 입장에서는 어떤 요금제가 가장 유리한지 판단하여라.

Table 6 Experimental settings for electricity unit costs

| Time             | UC 1 | UC 2 | UC 3 |
|------------------|------|------|------|
| Midnight – 10 am | 5    | 5    | 5    |
| 10 am – noon     | 10   | 10   | 10   |
| Noon – 4 pm      | 15   | 20   | 25   |
| 4 pm – 8 pm      | 10   | 10   | 10   |
| 8 pm - midnight  | 5    | 5    | 5    |

3. 기계 3을 생산용량이 16인 최신형 기계로 대체하였다. 새로운 생산계획을 수립하여라.  
(제조공전 전체의 Throughput이 바뀜)

# 과제 5 -1 답안

```
from ortools.linear_solver import pywraplp
```

```
UC = [0] + [5]*9 + [25]*7 + [5]*8
```

```
#UC1 = [0] + [5]*10 + [10]*2 + [15]*4 + [10]*4 + [5]*4
```

```
#UC2 = [0] + [5]*10 + [10]*2 + [20]*4 + [10]*4 + [5]*4
```

```
#UC3 = [0] + [5]*10 + [10]*2 + [25]*4 + [10]*4 + [5]*4
```

```
CAPA = [15, 14, 16, 16, 17, 16, 15, 18]
```

```
SC = 100 # setup cost
```

```
solver = pywraplp.Solver.CreateSolver("SAT")
```

```
# 생산량
```

```
P = {}
```

```
for i in range(8):
```

```
    for j in range(1, 25):
```

```
        P[i, j] = solver.NumVar(0, solver.infinity(), "P"+str(i)+'_'+str(j))
```

```
# 재고량
```

```
I = {}
```

```
for i in range(8):
```

```
    for j in range(25):
```

```
        I[i, j] = solver.NumVar(0, solver.infinity(), "I"+str(i)+'_'+str(j))
```

```
# 셋업비용 발생여부
```

```
S = {}
```

```
for i in range(8):
```

```
    for j in range(1, 25):
```

```
        S[i, j] = solver.IntVar(0, 1, "S"+str(i)+'_'+str(j))
```

```
# 기계 가동 여부
```

```
Y = {}
```

```
for i in range(8):
```

```
    for j in range(1,25):
```

```
        Y[i, j] = solver.IntVar(0, 1, "Y"+str(i)+'_'+str(j))
```

```
##### 재고제약
```

```
# 기계 0 - 6
```

```
for i in range(7):
```

```
    for j in range(1,25):
```

```
        solver.Add(I[i,j] - I[i,j-1] - P[i,j] + P[i+1,j] == 0, 'Inv_'+str(i)+str(j))
```

```
# 재고제약 기계 7
```

```
for j in range(1,25):
```

```
    solver.Add(I[7,j] - I[7,j-1] - P[7,j] == 0, 'Inv_7'+str(j))
```

```
##### 초기재고
```

```
# 기계 0-6
```

```
for i in range(7):
```

```
    solver.Add(I[i,0] - 2 * CAPA[i+1] == 0, 'init_inv_'+str(i)+'0')
```

```
#기계 7 초기 재고
```

```
solver.Add(I[7,0] == 0, 'init_inv_70')
```

```
##### 기말재고
```

```
# 기계 0 - 기계 6
```

```
for i in range(7):
```

```
    solver.Add(I[i,24] - 2 * CAPA[i+1] == 0, 'last_inv'+str(i)+'_24')
```

```
# 기계 7 (= Throughput)
```

```
solver.Add(I[7, 24] == 24*min(CAPA), 'last_inv_7_24')
```

# 과제 5 -1 답안

```
# 재고에 의한 생산량 제약
# 기계 0은 재고에 의한 생산 제약이 없음.
for i in range(1, 8):
    for j in range(1, 25):
        solver.Add(P[i,j] - I[i-1,j-1] <= 0, 'prod_inv_'+str(i)+str(j))

# 생산용량에 의한 제약 - 가동여부
for i in range(8):
    for j in range(1, 25):
        solver.Add(P[i,j] - CAPA[i] * Y[i,j] <= 0, 'prod_capa_'+str(i)+str(j))

for i in range(8):
    for j in range(1, 25):
        solver.Add(P[i,j] - Y[i,j] >= 0, 'prod_capa2_'+str(i)+str(j))

# 준비비용 발생 여부
for i in range(8):
    for j in range(2, 25):
        solver.Add(S[i,j] - Y[i,j] + Y[i,j-1] >= 0, 'setup_cost_'+str(i)+str(j))

# 첫 기간에 대한 준비 비용 발생여부
for i in range(8):
    #solver.Add(Y[i,0] == 0, 'setup_cost_'+str(i)+'0')
    solver.Add(S[i,1] - Y[i,1] == 0, 'setup_cost_'+str(i)+'1')

# Objective
objective_terms = []
for i in range(8):
    for j in range(1,25):
        objective_terms.append(UC[j] * P[i,j] + SC * S[i,j])
```

```
solver.Minimize(solver.Sum(objective_terms))

if 1:
    with open('or12-1.lp', "w") as out_f:
        lp_text = solver.ExportModelAsLpFormat(False)
        out_f.write(lp_text)

# Solve
status = solver.Solve()

p_data = []
i_data = []
DEBUG = 0
# Print solution.
if status == pywraplp.Solver.OPTIMAL or status == pywraplp.Solver.FEASIBLE:

    # 그래프 그리기 위한 데이터 정리
    for i in range(8):
        temp = []
        for j in range(1, 25):
            temp.append(P[i, j].solution_value())
        p_data.append(temp)

    for i in range(8):
        temp = []
        for j in range(25):
            temp.append(I[i, j].solution_value())
        i_data.append(temp)

    print(f"Total cost = {solver.Objective().Value():.1f}\n", )
else:
    print("No solution found.")
```

# 과제 5 - 2

- 다음 공정에 대하여 가변 전기 요금을 고려한 최적 운영 계획 문제를 푸시오.



- 가변적 전기요금은 다음과 같고 그 외에 모든 가정은 동일함.

|    | Utility Cost (UC <sub>k</sub> ) |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|---------------------------------|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 시간 | 1                               | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 |
| 가격 | 5                               | 5 | 5 | 5 | 5 | 5 | 10 | 10 | 10 | 15 | 15 | 15 | 15 | 25 | 25 | 25 | 15 | 15 | 15 | 10 | 10 | 10 | 5  | 5  |

## 과제 5 – 2 답안

```
from ortools.linear_solver import pywraplp
```

```
UC = [0] + [5]*6 + [10]*3 + [15]*4 + [25]*3 + [15]*3 + [10]*3 + [5]*2
CAPA = [12, 8, 15, 5, 15]
SC = 100 # setup cost
nMach = len(CAPA)
```

```
solver = pywraplp.Solver.CreateSolver("SAT")
```

```
# 생산량
```

```
P = {}
```

```
for i in range(nMach):
```

```
    for j in range(1, 25):
```

```
        P[i, j] = solver.NumVar(0, solver.infinity(), "P"+str(i)+'_'+str(j))
```

```
# 재고량
```

```
I = {}
```

```
for i in range(nMach):
```

```
    for j in range(25):
```

```
        I[i, j] = solver.NumVar(0, solver.infinity(), "I"+str(i)+'_'+str(j))
```

```
# 셋업비용 발생여부
```

```
S = {}
```

```
for i in range(nMach):
```

```
    for j in range(1, 25):
```

```
        S[i, j] = solver.IntVar(0, 1, "S"+str(i)+'_'+str(j))
```

```
# 기계 가동 여부
```

```
Y = {}
```

```
for i in range(nMach):
```

```
    for j in range(1, 25):
```

```
        Y[i, j] = solver.IntVar(0, 1, "Y"+str(i)+'_'+str(j))
```

```
##### 재고제약
```

```
# 기계 0 - 3
```

```
i = 0
```

```
for j in range(1,25):
```

```
    solver.Add(I[i,j] - I[i,j-1] - P[i,j] + P[i+1,j] == 0, 'Inv_'+str(i)+str(j))
```

```
i = 1
```

```
for j in range(1,25):
```

```
    solver.Add(I[i,j] - I[i,j-1] - P[i,j] + P[4,j] == 0, 'Inv_'+str(i)+str(j))
```

```
i = 2
```

```
for j in range(1,25):
```

```
    solver.Add(I[i,j] - I[i,j-1] - P[i,j] + P[i+1,j] == 0, 'Inv_'+str(i)+str(j))
```

```
i = 3
```

```
for j in range(1,25):
```

```
    solver.Add(I[i,j] - I[i,j-1] - P[i,j] + P[4,j] == 0, 'Inv_'+str(i)+str(j))
```

```
# 재고제약 기계 4
```

```
for j in range(1,25):
```

```
    solver.Add(I[4,j] - I[4,j-1] - P[4,j] == 0, 'Inv_4'+str(j))
```

```
##### 초기재고
```

```
# 기계 0-3
```

```
solver.Add(I[0,0] - 2 * CAPA[1] == 0, 'init_inv_0_0')
```

```
solver.Add(I[1,0] - 2 * CAPA[4] == 0, 'init_inv_1_0')
```

```
solver.Add(I[2,0] - 2 * CAPA[3] == 0, 'init_inv_2_0')
```

```
solver.Add(I[3,0] - 2 * CAPA[4] == 0, 'init_inv_3_0')
```

```
#기계 4 초기 재고
```

```
solver.Add(I[4,0] == 0, 'init_inv_4_0')
```

# 과제 5 – 2 답안

##### 기말재고

# 기계 0-3

solver.Add(I[0,24] - 2 \* CAPA[1] == 0, 'last\_inv\_0\_0')

solver.Add(I[1,24] - 2 \* CAPA[4] == 0, 'last\_inv\_1\_0')

solver.Add(I[2,24] - 2 \* CAPA[3] == 0, 'last\_inv\_2\_0')

solver.Add(I[3,24] - 2 \* CAPA[4] == 0, 'last\_inv\_3\_0')

# 기계 4 (= Throughput)

solver.Add(I[4, 24] == 120, 'last\_inv\_4\_24')

# 재고에 의한 생산량 제약

# 기계 0과 1은 재고에 의한 생산 제약이 없음.

for i in range(2, 4):

for j in range(1, 25):

solver.Add(P[i,j] - I[i-1,j-1] <= 0, 'prod\_inv\_'+str(i)+str(j))

i = 4

for j in range(1, 25):

solver.Add(P[i,j] - I[1,j-1] <= 0, 'prod\_inv\_1\_'+str(i)+str(j))

solver.Add(P[i,j] - I[3,j-1] <= 0, 'prod\_inv\_3\_'+str(i)+str(j))

# 생산용량에 의한 제약 - 가동여부

for i in range(5):

for j in range(1, 25):

solver.Add(P[i,j] - CAPA[i] \* Y[i,j] <= 0, 'prod\_capa\_'+str(i)+str(j))

for i in range(5):

for j in range(1, 25):

solver.Add(P[i,j] - Y[i,j] >= 0, 'prod\_capa2\_'+str(i)+str(j))

# 준비비용 발생 여부

for i in range(5):

for j in range(2, 25):

solver.Add(S[i,j] - Y[i,j] + Y[i,j-1] >= 0, 'setup\_cost\_'+str(i)+str(j))

# 첫 기간에 대한 준비 비용 발생여부

for i in range(5):

solver.Add(S[i,1] - Y[i,1] == 0, 'setup\_cost\_'+str(i)+'1')

# Objective

objective\_terms = []

for i in range(5):

for j in range(1,25):

objective\_terms.append(UC[j] \* P[i,j] + SC \* S[i,j])

#if i != 7 and j != 24:

# objective\_terms.append(IC \* I[i,j])

solver.Minimize(solver.Sum(objective\_terms))

if 1:

with open('or12-3.lp', "w") as out\_f:

lp\_text = solver.ExportModelAsLpFormat(False)

out\_f.write(lp\_text)

# Solve

status = solver.Solve()

## 과제 5 – 2 답안

```
p_data = []
i_data = []
DEBUG = 0
# Print solution.
if status == pywraplp.Solver.OPTIMAL or status == pywraplp.Solver.FEASIBLE:

    # 그래프 그리기 위한 데이터 정리
    for i in range(5):
        temp = []
        for j in range(1, 25):
            temp.append(P[i, j].solution_value())
        p_data.append(temp)

    for i in range(5):
        temp = []
        for j in range(25):
            temp.append(I[i, j].solution_value())
        i_data.append(temp)
```

```
if DEBUG == 0:
    for i in range(5):
        for j in range(1, 25):
            if P[i, j].solution_value() > 0.5:
                print("P[%d, %d] = %d" %(i, j, P[i, j].solution_value()))
elif DEBUG == 1:
    for i in range(5):
        for j in range(25):
            if I[i, j].solution_value() > 0.5:
                print("I[%d, %d] = %d" %(i, j, I[i, j].solution_value()))
else:
    for i in range(5):
        for j in range(1, 25):
            if Y[i, j].solution_value() > 0.5:
                print("Y[%d, %d] = %d" %(i, j, Y[i, j].solution_value()))
            print("S[%d, %d] = %d" %(i, j, S[i, j].solution_value()))

print(f"Total cost = {solver.Objective().Value():.1f}\n", )

import matplotlib.pyplot as plt

for i in range(5):
    plt.subplot(2, 4, i+1)
    plt.plot(p_data[i][1:], label = i)
    plt.legend()

for i in range(5):
    plt.subplot(2, 4, i+1)
    plt.plot(i_data[i][1:], label=i)
    plt.legend()

else:
    print("No solution found.")
```